

Báo cáo giải thuật bài toán Aura Tracker

LVT

Giới thiệu

Bài toán yêu cầu xác định tổng lượng Aura ghi nhận trong từng vùng theo dõi hình chữ nhật, mỗi Hunter chỉ được tính vào *vùng nhỏ nhất* chứa tọa độ của họ. Với số lượng vùng và số lượng Hunter lên đến 10^5 , bài toán cần một thuật toán tuyến tính–logarit tối ưu.

Dưới đây là trình bày cho từng Subtask theo đúng ba tiêu chí: (1) Phương pháp thiết kế thuật toán; (2) Lý do lựa chọn; (3) Phân tích độ phức tạp thời gian và bộ nhớ.

Subtask 1 (20 điểm): $n, m \leq 10^3$

1. Phương pháp thiết kế thuật toán

Phương pháp sử dụng: **duyệt brute force** Kiểm tra từng Hunter với tất cả hình chữ nhật, chọn vùng bao nhỏ nhất và cộng giá trị Aura tương ứng.

2. Tại sao phù hợp

Vì n, m đều nhỏ ($\leq 10^3$), tổng số phép kiểm tra tối đa chỉ là:

$$n \cdot m = 10^6$$

hoàn toàn trong giới hạn thời gian cho phép. Không cần cấu trúc dữ liệu phức tạp.

3. Độ phức tạp

Thời gian:

$$O(nm) = O(10^6)$$

Bộ nhớ: chỉ lưu danh sách hình chữ nhật và Hunter:

$$O(n + m)$$

Subtask 2 (30 điểm): $W, H \leq 10^3$

1. Phương pháp thiết kế thuật toán

Áp dụng **grid marking** + prefix sum hai chiều:

- Đánh dấu mỗi vùng hình chữ nhật lên một ma trận $W \times H$.
- Với mỗi ô trên lưới biết nó nằm trong vùng nhỏ nhất nào.
- Hunter chỉ cần truy cập trực tiếp ô (x, y) để biết thuộc vùng nào.

2. Tại sao phù hợp

Khi $W, H \leq 10^3$:

$$W \cdot H \leq 10^6$$

Hoàn toàn phù hợp để thao tác trên ma trận 2D. Thời gian và bộ nhớ đủ nhỏ để triển khai trực tiếp.

3. Độ phức tạp

Thời gian:

$$O(W \cdot H + n + m)$$

Bộ nhớ:

$$O(W \cdot H)$$

(lưu toàn bộ lưới kích thước tối đa 10^6 ô)

Subtask 3 (50 điểm): Không ràng buộc

1. Phương pháp thiết kế thuật toán

Sử dụng thuật toán quét theo trục (Sweep Line) kết hợp với cấu trúc dữ liệu tìm kiếm khoảng theo trục Y .

Cụ thể:

- Tạo các **event** theo hoành độ x :
 - Rectangle Entry (mở vùng)
 - Rectangle Exit (đóng vùng)
 - Query Point (Hunter)
- Duy trì một tập hợp các đoạn $[y_1, y_2]$ đang hoạt động được sắp xếp theo y_1 .
- Với mỗi Hunter, dùng **binary search** để tìm hình chữ nhật đang bao phủ nó theo trục y .

Mỗi Hunter chỉ bị tính vào vùng nhỏ nhất vì trong cùng một x tồn tại nhiều vùng lồng nhau, nhưng cấu trúc quét chỉ giữ đúng các vùng đang mở theo thứ tự.

2. Tại sao phù hợp

- Số lượng vùng và số Hunter lên đến 10^5 , không thể dùng brute force hay grid.
- Sweep Line là phương pháp tối ưu cho mọi bài toán giao cắt hình chữ nhật và điểm.
- Tất cả thao tác trên event đều có dạng:

insert, erase, binary search

đều xử lý được trong $O(\log n)$.

- Event sorting giúp giảm bài toán 2D thành bài toán 1D theo trục Y .

3. Độ phức tạp

Số lượng event:

$$2n + m \leq 3 \cdot 10^5$$

Thời gian:

$$O((n + m) \log(n + m))$$

bao gồm:

- sắp xếp event: $O((n + m) \log(n + m))$
- mỗi insert/remove/search: $O(\log n)$

Bộ nhớ:

$$O(n)$$

do chỉ lưu các khoảng Y của các hình chữ nhật đang mở.

Kết luận

Ba subtasks được giải bằng ba chiến lược hoàn toàn khác nhau:

- **Brute force** cho trường hợp nhỏ.
- **Grid + prefix sum** cho bản đồ nhỏ.
- **Sweep Line + interval search** cho bài toán tổng quát.

Thuật toán cuối cùng đạt yêu cầu tối ưu cho ràng buộc lớn nhất.